

Sesión 10

Implementar Tic Tac Toe

De pseudocódigo a código real. Traducir el diseño de la Sesión 9 a JavaScript manteniendo la separación lógica / UI.

IDEA CENTRAL

Implementar no es "escribir código que funcione".
Es **traducir decisiones de diseño a instrucciones** que el sistema ejecuta.
El diseño de la Sesión 9 es el mapa. Esta sesión es el camino.

1. Conexión con lo que ya sabemos

En la Sesión 9 diseñamos Tic Tac Toe sin escribir ni una línea de código. Definimos reglas, estado, casos borde y pseudocódigo. Ahora tenemos un mapa claro de **qué** tiene que pasar. Esta sesión responde al **cómo**.

Lo que ya tenemos en la app y reutilizamos directamente:

- **estado.js** — donde vive el estado global (pantallaActual, etc.)
- **app.js** — donde vive la lógica de la app y la función navegar()
- **index.html** — la SPA con todas las pantallas
- **styles.css** — apariencia, separada de la lógica
- El patrón: **evento** → **estado cambia** → **UI se actualiza**

PREGUNTA TENSION

¿Cómo pasamos del pseudocódigo que diseñamos la semana pasada a código JavaScript real, sin perder la claridad del diseño?

2. Dónde encaja Tic Tac Toe en el sistema

Tic Tac Toe no es un programa independiente. Es un **módulo** dentro de nuestra app. Tiene su propia lógica, pero usa el sistema que ya construimos.

Archivos nuevos:

- **ticTacToe.js** — la lógica del juego (reglas, turnos, victoria)
- **ticTacToeUI.js** — cómo se dibuja el tablero y se recogen clics

Archivos que ya existen y se reutilizan:

- **estado.js** — añadimos el estado del juego al objeto global
- **app.js** — conectamos la navegación con el inicio del juego
- **index.html** — añadimos el HTML del tablero dentro de la pantalla del juego

PRINCIPIO CLAVE

Separar en dos archivos **no es capricho**. Si mezclas reglas y dibujo en el mismo sitio, cuando cambies cómo se ve el tablero romperás la lógica del juego. Y viceversa.

3. Estado del juego — traducir el diseño

En la Sesión 9 definimos qué información necesita el juego. Ahora lo escribimos como JavaScript real:

```
// Dentro de estado.js
// Añadimos al estado global:

estado.ticTacToe = {
  tablero: [null, null, null,
            null, null, null,
            null, null, null],
  turno: 'X',
  ganador: null,
  terminado: false
}
```

Decisiones de diseño que ya tomamos:

- El tablero es un **array de 9 posiciones**, no una matriz 3x3. Más sencillo de recorrer.
- Cada posición vale **null** (vacía), 'X' o 'O'.
- **turno** sabe quién juega ahora.
- **ganador** empieza en null. Cuando alguien gana, se guarda 'X' u 'O'.
- **terminado** cubre tanto victoria como empate.

Correspondencia con el pseudocódigo:

Cada propiedad del estado tiene su equivalente directo en lo que diseñamos. Nada aparece "de la nada". Todo viene del diseño previo.

Pseudocódigo (Sesión 9)	JavaScript (Sesión 10)
tablero de 9 casillas	<code>tablero: [null x 9]</code>
turno actual	<code>turno: 'X'</code>
hay ganador?	<code>ganador: null</code>
partida acabada?	<code>terminado: false</code>

4. Lógica del juego — ticTacToe.js

Este archivo contiene las **reglas del juego**. No sabe nada de HTML, ni de botones, ni de colores. Solo sabe de estado.

4.1 Iniciar partida

```
function iniciarTicTacToe() {
  estado.ticTacToe = {
    tablero: [null, null, null,
              null, null, null,
              null, null, null],
    turno: 'X',
    ganador: null,
    terminado: false
  }
}
```

Resetear el estado es crear una partida nueva. No hay que borrar HTML ni limpiar la pantalla aquí — eso lo hará la UI.

4.2 Hacer un movimiento

```
function hacerMovimiento(posicion) {
  const juego = estado.ticTacToe

  // Caso borde: partida ya terminada
  if (juego.terminado) return

  // Caso borde: casilla ya ocupada
  if (juego.tablero[posicion] !== null) return

  // Colocar ficha
  juego.tablero[posicion] = juego.turno

  // Comprobar si hay ganador
  if (hayGanador(juego.tablero, juego.turno)) {
    juego.ganador = juego.turno
    juego.terminado = true
    return
  }

  // Comprobar empate
  if (tableroLleno(juego.tablero)) {
    juego.terminado = true
    return
  }

  // Cambiar turno
  juego.turno = juego.turno === 'X' ? 'O' : 'X'
}
```

Observa la estructura: primero los **casos borde** (protecciones), luego la acción principal, luego las comprobaciones de fin, y al final el cambio de turno. Es exactamente el orden del pseudocódigo.

4.3 Detectar ganador

```
function hayGanador(tablero, jugador) {
  const lineas = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8], // filas
    [0, 3, 6], [1, 4, 7], [2, 5, 8], // columnas
    [0, 4, 8], [2, 4, 6]             // diagonales
  ]
  for (const linea of lineas) {
    const [a, b, c] = linea
    if (tablero[a] === jugador &&
        tablero[b] === jugador &&
        tablero[c] === jugador) {
      return true
    }
  }
  return false
}
```

Todas las combinaciones ganadoras están en un array. Recorremos cada una y comprobamos si las tres casillas son del mismo jugador. No hay magia: es la tabla que hicimos en papel en la Sesión 9.

4.4 Detectar empate

```
function tableroLleno(tablero) {
  return tablero.every(casilla => casilla !== null)
}
```

Si no hay ganador y no quedan casillas vacías, es empate. Caso borde que ya identificamos en el diseño.

5. Interfaz del juego — ticTacToeUI.js

Este archivo se encarga de dos cosas: **dibujar** el estado del tablero y **escuchar** los clics del jugador. No toma decisiones sobre reglas.

5.1 Dibujar el tablero

```
function dibujarTablero() {
  const juego = estado.ticTacToe
  const contenedor = document.getElementById(
    'tablero-tictactoe'
  )
  // Limpiar y redibujar
  contenedor.innerHTML = ''
  juego.tablero.forEach((casilla, i) => {
    const celda = document.createElement('div')
    celda.className = 'celda'
    celda.textContent = casilla || ''
    celda.addEventListener('click', () => {
      hacerMovimiento(i)
      dibujarTablero() // re-dibujar tras movimiento
      mostrarMensaje()
    })
    contenedor.appendChild(celda)
  })
}
```

La UI lee el estado y lo refleja. Cuando el jugador hace clic, la UI llama a la lógica (`hacerMovimiento`), y luego se redibuja. Mismo patrón de siempre: evento → lógica → estado → UI.

5.2 Mostrar mensajes

```
function mostrarMensaje() {
  const juego = estado.ticTacToe
  const msg = document.getElementById('mensaje-ttt')
  if (juego.ganador) {
    msg.textContent = 'Gana ' + juego.ganador + '!'
  } else if (juego.terminado) {
    msg.textContent = 'Empate'
  } else {
    msg.textContent = 'Turno de ' + juego.turno
  }
}
```

6. HTML — lo que se añade a index.html

Dentro de la pantalla del juego que ya existe en nuestro SPA:

```
<div id="pantalla-juego-tic-tac-toe" class="pantalla">
  <h2>Tic Tac Toe</h2>
  <div id="mensaje-ttt">Turno de X</div>
  <div id="tablero-tictactoe" class="tablero"></div>
  <button onClick="reiniciarTicTacToe()">
    Nueva partida
  </button>
  <button onClick="navegar('menu')">
    Volver al menú
  </button>
</div>
```

El botón "Volver al menú" usa **navegar()**, la función que ya construimos. El sistema de navegación no cambia; el juego simplemente vive dentro de una pantalla.

7. Conectar todo — app.js

Cuando la app navega al juego, hay que iniciar la partida y dibujar el tablero:

```
// Dentro de actualizarUI() o al navegar:
if (estado.pantallaActual === 'juego-tic-tac-toe') {
  iniciarTicTacToe()
  dibujarTablero()
  mostrarMensaje()
}
```

Y para reiniciar:

```
function reiniciarTicTacToe() {
  iniciarTicTacToe()
  dibujarTablero()
  mostrarMensaje()
}
```

FLUJO COMPLETO

1. Usuario pulsa "Jugar Tic Tac Toe" en el menú
2. navegar('juego-tic-tac-toe') → cambia estado.pantallaActual
3. actualizarUI() → muestra la pantalla del juego
4. iniciarTicTacToe() → prepara estado del juego
5. dibujarTablero() → dibuja el tablero vacío
6. Cada clic → hacerMovimiento() → dibujarTablero()
7. Cuando termina → mostrarMensaje() muestra resultado

8. Mapa de archivos actualizado

```
mini-juegos-app/  
  index.html      // SPA - todas las pantallas  
  styles.css      // apariencia  
  estado.js       // estado global (incluye ticTacToe)  
  app.js          // navegacion, actualizarUI  
  ticTacToe.js    // logica del juego (NUEVO)  
  ticTacToeUI.js  // dibujar tablero (NUEVO)
```

Responsabilidades claras:

Archivo	Sabe de...	NO sabe de...
<code>ticTacToe.js</code>	reglas, turnos, victoria	HTML, botones, colores
<code>ticTacToeUI.js</code>	dibujar, escuchar clics	reglas, quién gana
<code>estado.js</code>	guardar datos	reglas ni dibujo
<code>app.js</code>	navegación, conectar	reglas de ningún juego

9. Uso de IA en esta sesión

Esta sesión es un momento ideal para usar IA como copiloto. Ya tenemos el diseño completo. Podemos pedirle que nos ayude a traducirlo a código — pero **nosotras validamos**.

Buenos prompts para la IA:

- "Tengo este pseudocódigo para Tic Tac Toe. Tradúcelo a JavaScript."
- "Revisa si mi función `hayGanador` cubre todas las combinaciones."
- "¿Qué pasa si el jugador hace clic en una casilla ocupada?"

Prompts peligrosos:

- "Hazme un Tic Tac Toe completo" — saltamos el diseño
- "Hazlo funcionar" — sin entender qué debe funcionar

Qué revisar del código que genere la IA:

- ¿Usa nuestros nombres? (`estado.ticTacToe`, no variables inventadas)
- ¿Separa lógica de UI? (no mezcla reglas con `document.getElementById`)
- ¿Cubre los casos borde? (casilla ocupada, partida terminada)
- ¿Se integra con `navegar()` y `actualizarUI()`?

10. Preguntas de pensamiento crítico

1. ¿Por qué `hacerMovimiento()` no dibuja nada en la pantalla? ¿Quién debería hacerlo?

2. Si quisiéramos añadir un tercer jugador (Z), ¿qué cambiaría en `ticTacToe.js`? ¿Y en `ticTacToeUI.js`?
3. ¿Qué pasaría si no tuviéramos el check `'if (juego.terminado) return'` al principio de `hacerMovimiento()`?
4. ¿Por qué el array de combinaciones ganadoras está dentro de `hayGanador()` y no en el estado?
5. Si borramos `ticTacToeUI.js`, ¿seguiría funcionando la lógica del juego? ¿Cómo lo comprobarías?

11. Mini-reto de la sesión (1 hora)

RETO: IMPLEMENTAR TIC TAC TOE

Usando el pseudocódigo de la Sesión 9 como guía, implementar el juego completo.

Paso 1 (10 min): Añadir el estado del juego a estado.js

Paso 2 (15 min): Escribir ticTacToe.js con las funciones de lógica

Paso 3 (15 min): Escribir ticTacToeUI.js para dibujar y escuchar clics

Paso 4 (10 min): Añadir HTML al index y conectar con app.js

Paso 5 (10 min): Probar casos borde: casilla ocupada, empate, ganar en diagonal

Criterio de éxito:

- El juego funciona: se puede jugar una partida completa
- La lógica y la UI están en archivos separados
- Los nombres coinciden con los del diseño (Sesión 9)
- Los casos borde están cubiertos
- Se puede volver al menú con navegar()

12. Qué NO hacer

Antipatrón	Por qué es problema
Meter lógica del juego en onclick del HTML	Mezcla estructura con lógica. Imposible de mantener.
Poner las reglas dentro de dibujarTablero()	UI y lógica mezcladas. Si cambias el diseño, rompes las reglas.
No cubrir casilla ocupada	Bug inmediato: se sobrescribe la ficha.
Crear un nuevo HTML para el juego	Rompe la SPA. Perdemos el estado y la navegación.
Copiar código de internet sin entenderlo	Probablemente no use nuestra arquitectura.
Ignorar el pseudocódigo y empezar de cero	Para eso no diseñamos. El diseño es el mapa.

FRASE DE CIERRE

**Implementar es traducir diseño a código, no inventar sobre la marcha.
Si el pseudocódigo estaba bien, el código sale casi solo.**

PREPARACIÓN SESIÓN 11

La próxima sesión diseñamos **Ahorcado** sin código. Mismo proceso: reglas, estado, casos borde, pseudocódigo. Pero esta vez comparando con Tic Tac Toe: ¿qué es igual? ¿qué cambia? ¿qué se reutiliza?